

Astaroth: A scientific computing framework for accelerating stencil computations.

Touko Puro^{1¶}, Johannes Pekkilä¹, Miikka S. Väisälä², Oskar Lappi³, Matthias Rheinhardt¹, Hsien Shang⁴, and Maarit Korpi-Lagg¹

¹ Aalto University, Finland ² University of Oulu, Finland ³ University of Helsinki, Finland ⁴ Academia Sinica Institute of Astronomy and Astrophysics, Taiwan ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Patrick Diehl](#)

Reviewers:

- [@rfj82982](#)
- [@streeve](#)
- [@atmyers](#)

Submitted: 20 May 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a

Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Stencil computations¹ are one of the bedrocks of high-performance scientific simulations, forming the core of many partial differential equation (PDE) and numerical linear algebra solvers. In recent years, GPUs have become the primary compute platform for data-parallel applications in high-performance computing, and it is difficult to run large simulations without them. Astaroth is a GPU framework for stencil computations, that has been developed to address this problem of scalable scientific computing. Astaroth provides its own domain specific language (DSL), in which researchers can express such computations without having to focus on technical implementation details. It can run efficiently both on CUDA- and HIP-based environments, and for testing purposes on CPUs. While stencils are the core of Astaroth, it also accelerates other operations like reductions (e.g. sums), simple ray-tracing, and integrates with libraries performing GPU-accelerated Fourier transforms, all of which are important for simulations on structured grids. Astaroth is optimized for multiphysics use cases and has primarily been used for turbulent astrophysical plasma simulations.

Statement of need

Much of the software used for scientific computing is written for CPUs, and has to be ported to GPUs to run larger problems with decent times-to-solution. Astaroth has been developed to solve this problem for the subset of scientific software that relies heavily on stencil computations and compared to existing solutions is specifically optimized for computations needed in multiphysics simulations. Astaroth's domain specific language (DSL) is designed to enable researchers to rewrite existing PDE solvers or to write completely new ones.

As examples of this, Astaroth has been used to write a partial differential equation (PDE) solver for astrophysical plasma simulations ([Väisälä et al., 2023](#)), which scales to thousands of GPUs with a weak scaling efficiency >90% [[Pekkilä \(2026\)](#)], and has also been used to accelerate the widely used astrophysics framework Pencil Code.

Of course, Astaroth's PDE solver is not limited to astrophysics, and neither is Astaroth limited to PDEs. As an example, many image processing techniques, like edge detection and convolutions, are stencil operations and stencil operations are widely used in numerical linear algebra.

¹Stencil computations are computations on structured grids where a given point is updated using a fixed neighborhood pattern. Examples are convolutions in image processing and convolutional neural networks, and different schemes for spatial derivatives like the finite-difference method.

37 State of the field

38 Several approaches have been proposed to improve the performance, portability, and
39 productivity dimensions of stencil computations. Single-process approaches include domain-
40 specific languages (DSLs), e.g., Halide (Ragan-Kelley et al., 2013), PolyMage (Mullapudi
41 et al., 2015), Delite (Sujeeth et al., 2014), and Lift (Steuer et al., 2017), and general
42 parallel processing abstractions, e.g., Kokkos (Trott et al., 2021) and RAJA (Beckingsale et al.,
43 2019). The primary benefit of a DSL is that assumptions about the structure of computations
44 can be made to improve the performance of generated code while maintaining a high-level
45 representation for the user. Performance-portability is typically augmented with automated
46 tuning and algorithm selection (e.g., PATUS (Christen et al., 2011), PARTANS (Lutz et al.,
47 2013)). These approaches are also adopted in Astaroth, which introduces a DSL, a code
48 generator, and implements automated tile-size optimization. A distinctive feature of Astaroth
49 is its algorithmic specialization for cache-constrained use cases in multiphysics, where the
50 working set required to update interdependent physical fields is too large to fit into on-chip
51 caches. This is addressed by reordering the computations into stages of higher and lower cache
52 reuse, thereby enabling better utilization of on-chip caches (Pekkilä et al., 2025). Astaroth
53 not only consider stencils in isolation, but also their combinations with other operations
54 inside the same kernel, such as the automatic fusion of distributed reductions with other
55 computations.

56 Methods to improve the performance-portability-productivity dimensions in multiprocess
57 applications include topology-aware workload distribution and abstractions for distributed
58 operations. Chapel (Callahan et al., 2004), Charm++ (Kalé & Krishnan, 1993) improve
59 the performance-portability-productivity dimensions by providing general-purpose distributed
60 programming models, whereas Cactus (Goodale et al., 2003), PETSc (Mills et al., 2021)
61 implement scientific computation toolkits. Further productivity advances can be gained with
62 domain-specialized frameworks, which provide ready-made solutions for specific use cases
63 and a simplified application programming interface. Astaroth also belongs to this class of
64 frameworks. Examples closest to Astaroth are Parthenon (Grete et al., 2023) and AMReX
65 (Zhang et al., 2019). Both are frameworks for distributed adaptive mesh refinement (AMR),
66 where Parthenon uses Kokkos as the compute backend and AMReX provides compute features
67 with parallel function wrappers and user-written C++ lambdas. In contrast to these projects,
68 Astaroth's distributed abstraction layer focuses on structured grid computations without mesh
69 refinement, and can thus make simplifying assumptions about the underlying data-movement
70 patterns to better address on-chip data movement, batching, and data-processing pipelines.
71 Similar to Parthenon and others (Pearson, 2021), Astaroth implements its modification of
72 topology-aware domain decomposition and rank reordering for improved portability across
73 systems, and performs fused packing to alleviate communication overheads. Furthermore,
74 Astaroth implements a task scheduler for compute and communication tasks (Lappi, 2021).

75 In the field, Astaroth stands out as a CUDA/HIP stencil-computing framework focused on
76 addressing the performance-productivity trade-off in cache-heavy multiphysics applications with
77 a domain-specific language, automated tile-size optimization, and topology handling, taking
78 ownership of data structures and movement throughout the computational science pipeline.
79 This enables holistic optimizations of full scientific workflows and offers a lower barrier of entry
80 to experimentation with optimization techniques spanning traditionally decoupled tasks (e.g.,
81 extensive kernel fusion of operations across the stack), which would not be practical with
82 libraries utilizing opaque submodules for compute and communication.

83 Software design

84 Astaroth consists of three main components: 1) acc, a compiler and runtime system for
85 a domain-specific language (DSL) for stencil computations, 2) an API for executing stencil
86 applications on multi-GPU platforms, and 3) a standalone PDE solver. Below, we present a

87 quick overview of these components. Extensive documentation is available at ([The Astaroth](#)
88 [developers, 2026](#)).

89 **acc compiler and runtime system**

90 Astaroth has a DSL for stencil-based computations, designed to be used by domain scientists
91 without having to consider technical implementation details. The main operations, such as
92 stencils, are written in a declarative syntax, and the kernels that use them are written in
93 an imperative syntax.² The implementation is left to Astaroth's DSL compiler *acc*, which
94 applies a number of specialized optimizations. An especially important optimization is the
95 unrolled and reordered computation of all required stencils at the start of the kernels, which
96 enables instruction-level parallelism and efficient usage of caches ([Pekkilä, 2026](#)). In addition
97 to stencils, the DSL supports two other operations: 1) multi-GPU reductions – which are
98 commonly needed for stencil-based solvers; and 2) simplified distributed ray-tracing, where
99 rays cannot change directions and are restricted to move through neighbouring grid points –
100 which is necessary for simulations incorporating radiative transfer ([Heinemann et al., 2006](#)).
101 Astaroth's DSL also includes a standard library, providing, inter alia: derivative operators used
102 in PDE solvers, implemented for generally spaced Cartesian, spherical or cylindrical grids; and
103 Poisson solvers, e.g. for self-gravity ([Krasnopolsky et al., 2026](#)).

104 *acc* transpiles the DSL source into CUDA or HIP source code, which is further compiled
105 into machine code using a native CUDA or HIP compiler. The program thus produced is
106 executed in the *acc* runtime system, which further optimizes the kernels by autotuning the
107 thread block sizes for kernel execution. *acc* also supports run-time compilation, because run-
108 time configuration parameters may change the evaluation of conditional statements, thereby
109 changing the branches taken at run-time. With run-time compilation, *acc* compiles for a given
110 configuration only those parts of the DSL source that will be executed. The information of
111 what code gets executed also allows Astaroth to optimize run-time behaviour more precisely,
112 e.g. memory allocations or communication patterns.

113 **Multi-GPU runtime system and API**

114 In the DSL, using the *ComputeSteps* language construct, users can define a list of compute steps
115 specifying a sequence of kernels and boundary conditions. Kernels defined in *ComputeSteps*
116 may be fused to reduce memory reads. Based on the overall domain decomposition and the
117 stencils' data access patterns, *acc* infers the dependency relationships between the steps, and
118 constructs a directed acyclic graph (DAG) of dependent tasks. Each step is split into many
119 tasks, one per region: there are 26 smaller regions at each subdomain's boundaries, which
120 are dependent on communicated data from neighbors; and one big region at the core of the
121 subdomain, which is not. Communication tasks are inserted where needed.

122 Astaroth's task scheduler executes these DAGs, asynchronously launching computation and
123 communication tasks when prerequisite tasks are completed. This improves performance in
124 communication-bound cases, especially for higher process counts ([Lappi, 2021](#)). For fast data
125 transfers and to support all possible hardware, both GPU-to-GPU remote direct memory access
126 (RDMA) and CPU-to-CPU communication are supported.

127 This runtime system can be accessed through Astaroth's runtime API. The API is C-ABI
128 compatible, supporting foreign function interfaces to external applications written in any
129 programming language. The API is organized into two layers: the *Device* layer and the *Grid*
130 layer. The *Device* layer provides access to single-GPU functionality, such as: moving data
131 between CPU and GPU, launching kernels, and loading/storing snapshots from/to disk. The
132 *Grid* layer provides access to multi-GPU functionality, such as: executing DAGs, distributed

²In declarative programming, computations are defined by describing what the results look like; in imperative programming, by describing the steps to perform.

133 initialization, and distributed loading/storing of snapshots. Other special functionality is also
134 provided through the API, such as distributed Fourier transforms.

135 Solver

136 Astaroth also includes a standalone finite-difference PDE solver (Pekkilä et al., 2022), which
137 takes full advantage of the DSL and the multi-GPU API, and can be used to write new
138 simulation models. It also works as a testbed for performance research. This solver uses an
139 astrophysical magnetohydrodynamical setup (acc-runtime/samples/mhd_modular) by default,
140 but can be configured to run any DSL code. The samples directory also includes other
141 production-ready setups, e.g. tfm-mpi for the test-field method (Pekkilä, 2026).

142 The solver handles distributed initial conditions, domain decomposition, simulation diagnostics,
143 and logging. It is also designed to react to a number of events, such as NaNs in the simulation
144 data, simulation time limits, and a stop signal given through the file system. The directory
145 analysis/ contains Python-based data analysis tools, which can be used to process and work
146 with the data produced by the standalone solver.

147 Research impact statement

148 Astaroth has already been used in a number of papers as the core PDE-solver, mainly for
149 astrophysical plasma simulations (Gent et al., 2026; Väisälä et al., 2021, 2023), but also in
150 seismology (Ladino et al., 2025). Additionally, it has been used for research on performance
151 optimization methods (Pekkilä et al., 2017, 2025; Pekkilä, 2026), communication techniques
152 (Lappi, 2021; Pekkilä et al., 2022), compiler techniques (Pekkilä, 2019; Puro, 2023) and
153 other topics (Puro & Pohjonen, 2025; Yokelson et al., 2024). We expect that the recent
154 GPU-acceleration of Pencil Code, which was done by embedding Astaroth's DSL and runtime
155 system into it, will increase the number of Astaroth users. The associated speedup factor
156 of 20-60 (Pekkilä et al., 2022) will enable more realistic astrophysical simulations in a wide
157 range of use cases from modelling small-scale dynamos (Warnecke et al., 2025) to processes
158 producing primordial gravitational waves and their propagation (Roper Pol et al., 2020).

159 Acknowledgements

160 We acknowledge the contributions of all developers and early users of Astaroth who have
161 been instrumental in its evolution. These include Petr Bém, Jörn Warnecke, Frederick Gent,
162 Ruben Krasnopolsky, Wei-Wen Li, Mordecai Mac Low, Chun-Fan Liu, Man Hei Li, Tzu-Chun
163 Hsu and Indrani Das. We acknowledge the computational resources and services provided
164 by CSC — IT Center for Science, the Aalto Science-IT project, ASIAA High-Performance
165 Computing, and National Center for High-Performance Computing (NCHC), National Applied
166 Research Laboratories (NARLabs) in Taiwan, the Oak Ridge Leadership Computing Facility
167 at the Oak Ridge National Laboratory, and resources from LUMI-G through the Euro-HPC
168 joint undertaking. Furthermore, we appreciate the important technical assistance provided by
169 CSC, by people like Fredrik Robertsén and others. The development of Astaroth has received
170 funding from the Academy of Finland, ReSolVE Centre of Excellence, Grant/Award Number:
171 307411; The European Research Council, the European Union's Horizon 2020 research and
172 innovation program, project UniSDyn, Grant/Award Number: 818665; KAUTE Foundation,
173 Grant/Award Numbers: 20240173 and 20250154; Research Council of Finland, project MomEnt,
174 Grant/Award Number: 373416. The authors acknowledge support for the CompAS Project
175 from the Institute of Astronomy and Astrophysics, Academia Sinica (ASIAA), the Academia
176 Sinica grant AS-IAIA-114-M01, and the National Science and Technology Council (NSTC)
177 in Taiwan through grants 112-2112-M-001-030, 113-2112-M-001-008, and 114-2112-M-001-
178 001-; the International Collaboration and Cooperation grant for COSMAGG that supports
179 the exchanges between Taiwan and Finland: 113-2927-I-001-513-, 114-2927-I-001-506-, and

Research Council of Finland project 359462. MSV thanks the support of Jenny and Antti Wihuri Foundation and Finnish Cultural Foundation in his doctoral thesis work, and therefore the early development in Astaroth prototype.

AI usage disclosure

AI tools have not been used in any step of software creation, documentation or in the authoring of this paper.

References

- Beckingsale, D. A., Burmark, J., Hornung, R., Jones, H., Killian, W., Kunen, A. J., Pearce, O., Robinson, P., Ryujin, B. S., & Scogland, T. R. (2019). RAJA: Portable performance for large-scale scientific applications. *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, 71–81. <https://doi.org/10.1109/p3hpc49587.2019.00012>
- Callahan, D., Chamberlain, B. L., & Zima, H. P. (2004). The cascade high productivity language. *Proceedings of the Ninth International Workshop on High-Level Parallel Programming Models and Supportive Environments, 2004. Proceedings.*, 52–60. <https://doi.org/10.1109/HIPS.2004.1299190>
- Christen, M., Schenk, O., & Burkhart, H. (2011). PATUS: A code generation and autotuning framework for parallel iterative stencil computations on modern microarchitectures. *Proceedings of the 2011 IEEE International Parallel & Distributed Processing Symposium*, 676–687. <https://doi.org/10.1109/IPDPS.2011.70>
- Gent, F. A., Mac Low, M.-M., Korpi-Lagg, M. J., Puro, T., & Rheinhardt, M. (2026). Asymptotic behaviour of galactic small-scale dynamos at modest magnetic prandtl number. *The Astrophysical Journal Letters*, 1000(2), L40. <https://doi.org/10.3847/2041-8213/ae4c41>
- Goodale, T., Allen, G., Lanfermann, G., Massó, J., Radke, T., Seidel, E., & Shalf, J. (2003). The cactus framework and toolkit: Design and applications. In G. Goos, J. Hartmanis, J. Van Leeuwen, J. M. L. M. Palma, A. A. Sousa, J. Dongarra, & V. Hernández (Eds.), *High performance computing for computational science — VECPAR 2002* (pp. 197–227). Springer. https://doi.org/10.1007/3-540-36569-9_13
- Grete, P., Dolence, J. C., Miller, J. M., Brown, J., Ryan, B., Gaspar, A., Glines, F., Swaminarayan, S., Lippuner, J., Solomon, C. J., Shipman, G., Junghans, C., Holladay, D., Stone, J. M., & Roberts, L. F. (2023). Parthenon—a performance portable block-structured adaptive mesh refinement framework. *The International Journal of High Performance Computing Applications*, 37(5), 465–486. <https://doi.org/10.1177/10943420221143775>
- Heinemann, T., Dobler, W., Nordlund, Å., & Brandenburg, A. (2006). Radiative transfer in decomposed domains. *Astronomy & Astrophysics*, 448(2), 731–737. <https://doi.org/10.1051/0004-6361:20053120>
- Kalé, L. V., & Krishnan, S. (1993). CHARM++: A portable concurrent object oriented system based on C++. *Proceedings of the Eighth Annual Conference on Object-Oriented Programming Systems, Languages, and Applications*, 91–108. <https://doi.org/10.1145/165854.165874>
- Krasnopolsky, R., Puro, T., Li, W.-W., Shang, H., Väisälä, M. S., Mac Low, M.-M., Rheinhardt, M., & Korpi-Lagg, M. (2026). Iterative poisson solvers for self-gravity with the GPU code astaroth. *arXiv Preprint arXiv:2605.03563*. <https://doi.org/10.48550/arXiv.2605.03563>
- Ladino, O., Coelho, B. S., Wyzykowski, A. B. V., & Soares, L. (2025). Acoustic wave

- modeling using the astaroth framework. *Empowering the Energy Shift: The Role of HPC in Sustainable Innovation, 2025*, 1–3. <https://doi.org/10.3997/2214-4609.2025643013>
- Lappi, O. (2021). A task scheduler for astaroth, the astrophysics simulation framework. *Master's Thesis*. <https://urn.fi/URN:NBN:fi-fe2021050428868>
- Lutz, T., Fensch, C., & Cole, M. (2013). PARTANS: An autotuning framework for stencil computation on multi-GPU systems. *ACM Transactions on Architecture and Code Optimization*, 9(4), 59:1–59:24. <https://doi.org/10.1145/2400682.2400718>
- Mills, R. T., Adams, M. F., Balay, S., Brown, J., Dener, A., Knepley, M., Kruger, S. E., Morgan, H., Munson, T., Rupp, K., Smith, B. F., Zampini, S., Zhang, H., & Zhang, J. (2021). Toward performance-portable PETSc for GPU-based exascale systems. *Parallel Computing*, 108, 102831. <https://doi.org/10.1016/j.parco.2021.102831>
- Mullapudi, R. T., Vasista, V., & Bondhugula, U. (2015). Polymage: Automatic optimization for image processing pipelines. *ACM SIGARCH Computer Architecture News*, 43(1), 429–443. <https://doi.org/10.1145/2694344.2694364>
- Pearson, C. (2021). *Movement and placement of non-contiguous data in distributed GPU computing* [PhD thesis, University of Illinois at Urbana-Champaign]. <https://hdl.handle.net/2142/110511>
- Pekkilä, J. (2019). Astaroth: A library for stencil computations on graphics processing units. *Master's Thesis*. <https://urn.fi/URN:NBN:fi:aalto-201906233993>
- Pekkilä, J. (2026). *Graphics processors in high-performance computing: Practice and experience in the simulation of astrophysical magnetohydrodynamics and turbulence* [PhD thesis, Aalto University School of Science]. <https://urn.fi/URN:ISBN:978-952-64-3160-4>
- Pekkilä, J., Lappi, O., Robertsén, F., & Korpi-Lagg, M. J. (2025). Stencil computations on amd and nvidia graphics processors: Performance and tuning strategies. *Concurrency and Computation: Practice and Experience*, 37(12-14), e70129. <https://doi.org/10.1002/cpe.70129>
- Pekkilä, J., Väisälä, M. S., Käpylä, M. J., Käpylä, P. J., & Anjum, O. (2017). Methods for compressible fluid simulation on GPUs using high-order finite differences. *Computer Physics Communications*, 217, 11–22. <https://doi.org/10.1016/j.cpc.2017.03.011>
- Pekkilä, J., Väisälä, M. S., Käpylä, M. J., Rheinhardt, M., & Lappi, O. (2022). Scalable communication for high-order stencil computations using CUDA-aware MPI. *Parallel Computing*, 111, 102904. <https://doi.org/10.1016/j.parco.2022.102904>
- Puro, T. (2023). Programmatic and efficient gpu acceleration of cpu stencil codes. *Master's Thesis*. <http://hdl.handle.net/10138/569388>
- Puro, T., & Pohjonen, A. (2025). GPU acceleration of average gradient method for solving partial differential equations. *Proceedings of the Second SIMS EUROSIM Conference on Modelling and Simulation, SIMS EUROSIM 2024*. <https://doi.org/10.3384/ecp212.066>
- Ragan-Kelley, J., Barnes, C., Adams, A., Paris, S., Durand, F., & Amarasinghe, S. (2013). Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6), 519–530. <https://doi.org/10.1145/2491956.2462176>
- Roper Pol, A., Mandal, S., Brandenburg, A., Kahnishvili, T., & Kosowsky, A. (2020). Numerical simulations of gravitational waves from early-universe turbulence. *Physical Review D*, 102(8), 083512. <https://doi.org/10.1103/physrevd.102.083512>
- Steuwer, M., Rummel, T., & Dubach, C. (2017). LIFT: A functional data-parallel IR for high-performance GPU code generation. *Proceedings of the 2017 IEEE/ACM International Symposium on Code Generation and Optimization*, 74–85. <https://doi.org/10.1109/CGO.2017.0000000>

272 [2017.7863730](#)

273 Sujeeth, A. K., Brown, K. J., Lee, H., Rompf, T., Chafi, H., Odersky, M., & Olukotun,
274 K. (2014). Delite: A compiler architecture for performance-oriented embedded domain-
275 specific languages. *ACM Transactions on Embedded Computing Systems*, 13(4s), 1–25.
276 <https://doi.org/10.1145/2584665>

277 The Astaroth developers. (2026). *Astaroth documentation*. [https://toxypuro.github.io/](https://toxypuro.github.io/astaroth/)
278 [astaroth/](https://toxypuro.github.io/astaroth). <https://toxypuro.github.io/astaroth>

279 Trott, C. R., Lebrun-Grandié, D., Arndt, D., Ciesko, J., Dang, V., Ellingwood, N., Gayatri, R.,
280 Harvey, E., Hollman, D. S., Ibanez, D., & others. (2021). Kokkos 3: Programming model
281 extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems*,
282 33(4), 805–817. <https://doi.org/10.1109/tpds.2021.3097283>

283 Väisälä, M. S., Pekkilä, J., Käpylä, M. J., Rheinhardt, M., Shang, H., & Krasnopolsky, R.
284 (2021). Interaction of large-and small-scale dynamos in isotropic turbulent flows from
285 GPU-accelerated simulations. *The Astrophysical Journal*, 907(2), 83. [https://doi.org/10.](https://doi.org/10.3847/1538-4357/abceca)
286 [3847/1538-4357/abceca](https://doi.org/10.3847/1538-4357/abceca)

287 Väisälä, M. S., Shang, H., Galli, D., Lizano, S., & Krasnopolsky, R. (2023). Exploring the
288 formation of resistive pseudodisks with the GPU code astaroth. *The Astrophysical Journal*,
289 959(1), 32. <https://doi.org/10.3847/1538-4357/acfb00>

290 Warnecke, J., Korpi-Lagg, M. J., Rheinhardt, M., Viviani, M., & Prabhu, A. (2025). Small-scale
291 and large-scale dynamos in global convection simulations of solar-like stars. *Astronomy &*
292 *Astrophysics*, 696, A93. <https://doi.org/10.1051/0004-6361/202451085>

293 Yokelson, D., Lappi, O., Ramesh, S., Väisälä, M. S., Huck, K., Puro, T., Norris, B., Korpi-Lagg,
294 M., Heljanko, K., & Malony, A. D. (2024). SOMA: Observability, monitoring, and in situ
295 analytics for exascale applications. *Concurrency and Computation: Practice and Experience*,
296 36(19), e8141. <https://doi.org/10.1002/cpe.8141>

297 Zhang, W., Almgren, A., Beckner, V., Bell, J., Blaschke, J., Chan, C., Day, M., Friesen, B.,
298 Gott, K., Graves, D., Katz, M., Myers, A., Nguyen, T., Nonaka, A., Rosso, M., Williams, S.,
299 & Zingale, M. (2019). AMReX: A framework for block-structured adaptive mesh refinement.
300 *Journal of Open Source Software*, 4(37), 1370. <https://doi.org/10.21105/joss.01370>